Migration guide for Easy Idle Game 2.0

This guide covers projects updating from Easy Idle Game 1.7.2 to 2.0. Before importing the update, commit the project to version control (or make a full project backup) and keep copies of representative player saves: importing overwrites files inside the Easy Idle Game installation folder, including any modifications you made there, and a pre-update commit is the only reliable way back. Use Breaking Changes (`EasyIdleGame > Documentation > md > BreakingChanges.md`) for the exhaustive API table.

Table of contents

Source/API migrations

Old	Current	Required action
`UpgradeType.cost`	`UpgradeType.purchaseCost`	Rename source references. Serialized enum value remains `2`; use the specific contextual cost types where appropriate.
`IValidatable`, `Business.GetValidationErrors()`	Inspector/custom validation	Remove the interface/calls; add project runtime validation when data can bypass the Inspector.
`Business.outputs`, `IProducable.Outputs` authoring	`outputTables`, `OutputTables`	Author tables and evaluation mode. `Outputs` is only a flattened compatibility view.
direct reward arrays	`rewardTables`, `prestigeRewardTables`	Preview with average/expected APIs and claim through system methods.
`ApplyOuputs(...)`	`ApplyOutputs(BigNumber)`	Correct spelling and handle returned results.
`GetActualProduction(...)` for preview	optimistic preview methods	Actual calls may roll; never use them merely to render UI.
`ClaimDailyRewards(day)`	`TryClaimDailyRewards(day, out outputs)`	Handle failure and returned applied rewards.
`PrestigeGame()`	`TryPrestigeGame(...)`	Check success; use outputs and optional `ResetSummary`.
`StartProduction()` as primary call	`TryStartProduction`, `TryProduceManually`, contextual `StartProduction`	Treat failure as normal.
single `producing`/`productionTime`	`ActiveProductions`, `CurrentlyProducing`, progress helpers	Update concurrent/snapshot/live-round state handling.

`OnProductionRoundFinished()`	`OnProductionRoundFinished(ProductionRound)`	Keep the event round and its rolled outputs.
void `ClaimOutputs()`	returned `List<Output>`	Use results for UI, analytics, and synchronization.
string auto-produce/group targeting	business and typed `BusinessGroup` assets	Replace legacy strings; pass the target business.
merge holder input/output arrays	`inputs`, `outputTables`	Migrate definitions and handle returned outputs.
runtime recipes auto-discovered	cached `Resources.LoadAll`	Keep recipes under `Resources`; call `RefreshMergeRecipeCache()` after runtime changes.
manager convenience calls	manager interfaces/current typed APIs	Use `iBuyable`, `iUnlockable`, `iLevelupable`, typed production queries.
`UpgradeHolder.OnBought`	`ApplyBoughtAmount(amount, source)`	Keep timed acquisition and payment separate; prefer manager purchase calls.
source-less holder additions	`SourceType` overloads	Pass purchase, ad, production, milestone, reset, conversion, etc.

Behavior changes

Current, total, and bought amounts

`Amount` is current state, `TotalAmount` is lifetime acquisition, and `BoughtAmount` contains purchase-flow acquisition. With `UseGeometricBoughtAmount`, grants do not inflate future prices. Normal free-grant APIs may still respect capacity.

Production rounds

Outputs roll at round start. New owned copies join or wait according to `ProductionRoundMode`; concurrent mode has independent rounds. Costs, partial production, consume-on-finish, production locks, previews, per-second, and offline calculations now have distinct paths/contexts.

Random rewards

`DropStrategy.All` evaluates independent entries; `WeightedRandom` makes weighted picks. `EvaluateOnce` rolls once and scales; `EvaluatePerInstance` evaluates a batch and uses expected-value fallback for large amounts. A weighted table without positive eligible weight grants nothing.

Contextual upgrades

Filters can match source/target assets and groups, output target, levels, manager/boost/upgrade/recipe, and manual/offline state. Custom previews must pass the same context as real operations.

Typed groups and capacity

All major item families have typed group assets. Business and currency groups can impose shared caps; relevant groups target locks/effects/offers/prestige. Legacy strings are fallback-only.

Tiered achievements and scoped prestige

Achievements may have multiple scaled claims. Prestige can reset locations/achievements, preserve explicit assets or groups, accept per-call exclusions, and return a summary.

Applied profit and external purchases

Offline application returns actual accepted amounts after caps/stockpiles. Ads/real-money shop attempts delegate only; project code must call completion after verified provider success.

Serialized asset migration

Unity compatibility code migrates legacy production/reward/merge arrays and metadata fields. It does not prove the result is designed correctly. Inspect and resave:

- output strategy/evaluation, chance, weight, and min/max amounts;
- metadata and asset identity;
- typed groups and shared capacity;
- lower/upper and production locks;
- intentionally empty input/output tables;
- wrapper self-reference rewriting and scales.

`BusinessWrapper.SyncData()` overwrites copied fields during enable/validation. Put supported differences in wrapper metadata/scales or create a standalone business.

Resources identifiers and save-field seeding

Persisted definitions must remain below a `Resources` folder and keep a unique, stable same-type asset name. Changing only `Metadata.name` is safe; renaming the asset itself changes its save identifier. If a rename is unavoidable, migrate stored JSON identifiers or provide a temporary compatibility loader and verify representative saves before removing it.

Older saves may omit newer counters. `SaveFile.Load()` currently seeds:

- Currency `totalAmount` from current amount when total is zero.
- Business `totalAmount` from current amount when total is zero.
- Business `boughtAmount` from total amount when bought is zero.
- Manager, location, shop, and upgrade bought counts from their older available amounts.
- Boost bought amount from its total amount.

These are conservative assumptions. Historical free grants may become counted as bought. Use a versioned project migration when that affects pricing or analytics.

Save compatibility

New base saves include locations, shop items, achievements, custom stats, active boosts, expanded daily rewards, and total/bought amounts. Missing totals/bought values are seeded from compatible older fields.

Custom save systems must preserve `base.Load()`, include new base fields, suppress dirty callbacks during load, clamp offline time, apply profit after base state, and reconcile physical scene state. Test both an untouched pre-update save and a post-migration save.

Validated runtime constraints

Include these cases in migration testing:

- Load existing state before the enabled `SaveAndLoad` component reaches its first `Update` auto-save.
- Verify legacy saves migrate to the new `Path.Combine`-based save location on first `Awake()` / `Load()`.
- Choose one shop reward policy; built-in completion grants immediately and `TryConsumeItem` grants again.
- Bulk shop completions are floored and clamped to the remaining purchase limit by the package.
- Daily reward claims reject non-positive, future, already-claimed, and rewardless days; non-positive repeating divisors are skipped.
- Verify offline boost/manager timer behavior for the time-skip policies your game enables.
- Add every manager referenced by a lock or output. Configured lock requirements now throw an `InvalidOperationException` naming any missing manager.

- Rebalance merge cooldowns: they now follow `UpgradeType.cooldown` blocks (multiplicative) instead of the merge speed multiplier.

Search checklist

Review (do not blindly replace) hits for:

```
UpgradeType.cost
IValidatable
GetValidationErrors
ApplyOuputs
GetActualProduction(
ClaimDailyRewards
PrestigeGame(
GetBusinessAmount
AddBusinesses
IsBusinessUnlocked
GetCurrentItemLevel
GetTotalProductionPerSecondOfType
GetUpgradeAmount
IsBoostUnlocked
BoostsManager.IsAutoProduceOverrideActive(
UpgradeHolder.OnBought
BusinessMergeRecipe.input
BusinessMergeRecipe.output
BusinessMergeRecipeHolder
producing
productionTime
outputs
rewards
```

Some names remain as obsolete/serialization shims or valid local variables.

Verification matrix

Area	Feature demo
Core loop	`01_BasicFlow`
Costs, totals, caps, locks	`LocksAndBuyAmounts`
Production rounds/claims	`ProductionAndCollection`
Input-limited production	`ProductionCosts`
Random rewards	`DropTableRewards`
Contextual effects/groups	`BoostsAndUpgrades`, `BusinessGroupTargeting`
Merge migration	`MergingRecipes`
Save/offline/cap application	`OfflineProfitApplication`
New managers	`LocationTravel`, `ShopRewardCompletion`, `PlayerProgression`, `DailyRewards`, `PrestigeResets`

Recommended migration order

1. Resolve Unity/package compatibility and compile enum/API renames.

2. Update custom interface implementations.
3. Inspect and resave migrated ScriptableObjects.
4. Replace string groups and validate shared capacities.
5. Handle success flags, output lists, summaries, and source types.
6. Update previews to optimistic/expected/contextual APIs.
7. Load a copy of an old production save and verify totals/bought amounts.
8. Run the closest feature demos and project integration tests.
9. Consult the maintained Markdown scripting reference and current source for the current API surface.

Retiring compatibility data

Only after every supported asset and save has migrated:

1. Stop authoring legacy fields and string groups.
2. Remove project code that calls obsolete members.
3. Keep released-save fixtures for every supported migration boundary.
4. Document the minimum supported source version.
5. Remove compatibility shims only in a deliberate breaking release.